

PRIMORDIA

A 3D Emergence Laboratory

Two-scale particle life with Barnes–Hut gravity and a piloted articulated creature

Technical Report and User Manual • Interactive Graphics course

Type	Real-time interactive 3D web app	External assets	None (all procedural)
Environment	WebGL via three.js r128, ES6 JS	Entry point	index.html

Abstract. PRIMORDIA is a real-time laboratory for *emergence*: thousands of particles obey two simple force laws acting at deliberately different spatial scales, and the structures that follow — cells, filaments, worms, spirals, orbiting discs — are never programmed explicitly. The user does not build them; the user edits the rules, then watches, perturbs and swims through whatever the rules produce. This document is both a technical presentation and a user manual: Sections 2–5 cover the environment, the external dependencies and every technical aspect of the implementation, Section 6 documents the implemented interactions, and Section 7 can be read on its own by someone who only wants to run the application.

1 Introduction

The simulation superimposes two force laws, which is the conceptual core of the project.

- **Short-range “chemistry”.** Every particle carries a discrete *type* (a colour). An *asymmetric* matrix A specifies how strongly type i is attracted to or repelled by type j within a small radius $r_{\max}$. Because $A_{ij} \neq A_{ji}$ in general the system is non-conservative, which produces the chasing and self-propelling behaviour characteristic of *particle life*. This term builds local structure: cells, membranes, filaments, worms.
- **Long-range gravity.** Every particle also carries a mass derived from its radius and attracts every other particle with a softened inverse-square law. This term builds global structure: collapse, clumping, orbits, spiral arms.

Neither force alone is novel; their superposition is what makes the application worth exploring. Gravity assembles the coarse skeleton, chemistry decorates it, and the two fight each other in ways that are hard to predict from the parameter values. Naive evaluation of both laws costs $O(N^2)$ per frame, far too slow for the 1000–3000 particles we target at 60 Hz, so each law is paired with the acceleration structure that suits its scale: a uniform grid for the short-range term, a Barnes–Hut octree for the long-range one. Finally, the soup is inhabited: a hierarchical self-animated creature (the *Grazer*) with four articulated legs is piloted by the user, is a solid obstacle particles bounce off, and emits its own force field — a physical participant, not a decorative overlay.

Design goals. (i) **Everything procedural** — no mesh, image or animation files; every vertex, texel and joint angle is computed in JavaScript at run time. (ii) **Real-time at meaningful N** , which rules out all-pairs evaluation. (iii) **Everything live** — no parameter change ever requires a reload; particle count, type count, world size and the rule matrix are all edited while the simulation runs. (iv) **The rules are visible** — the octree subdivision, the gravitational field and a deforming lattice of space can all be displayed; the acceleration structure is part of the exhibit rather than hidden behind it. (v) **Zero-friction deployment** — opening [index.html](#) from disk must be enough: no build step, no bundler, no server, no `npm install`.

2 Description of the environment used

The application is a **client-side web application** rendering through **WebGL**, accessed via the **three.js** library (r128) rather than through raw WebGL calls. The context is created by `THREE.WebGLRenderer`, which selects WebGL2 when available and silently falls back to WebGL1; we use no WebGL 2-only feature, so both paths are correct. The renderer runs with `antialias:true` (MSAA on the default framebuffer) and `powerPreference: 'high-performance'` (asks for the discrete GPU on dual-GPU laptops). The device pixel ratio is clamped with `setPixelRatio(Math.min(devicePixelRatio,2))`: unclamped, a high-DPI display would quadruple the fragment load for a barely perceptible gain. Output is written in sRGB (`outputEncoding = THREE.sRGBEncoding`) so lighting is composited in linear space and converted once at the end, which is what makes the emissive teals and pinks read correctly instead of washing out.

We chose three.js over hand-written WebGL deliberately: the intellectual content of the project is the *simulation* — force laws, grid, octree, articulated model, gait — and all of that is written from scratch, so

delegating context creation, shader compilation and matrix bookkeeping to a mature library let us spend the available time on the parts that are actually ours. `three.js` is a rendering back end and a scene-graph container here, never a simulation framework.

Language and execution model. Plain **ES6+ JavaScript**: no TypeScript, no JSX, no transpilation, no bundling, so the source that ships is the source that runs. The files are loaded as *classic scripts* (`<script src=...>` in dependency order, one global lexical scope) rather than ES modules — a conscious trade-off, since browsers refuse to load modules over `file://`, which would force every user to start a local server and would break the “double-click `index.html`” promise. Everything runs single-threaded inside the animation-frame callback; there are no Web Workers, which bounds the achievable particle count (Section 8).

Requirement	Detail
Browser / GPU	Any modern browser with WebGL (Chrome, Firefox, Edge, Safari); integrated graphics suffice
Network	Needed <i>once</i> , to fetch <code>three.js</code> from the CDN (§3)
Server / installation	Not required — the <code>file://</code> protocol works; nothing to install
Input	Mouse + keyboard (full feature set); touch (camera only)

The layout is responsive: below 560px of viewport width the console expands to full width and the HUD is hidden, and the camera accepts one-finger drag (orbit) and two-finger pinch (zoom). Piloting and the drag tools need a keyboard and a mouse, so a desktop browser is the intended target.

3 External libraries, tools and models

This section lists exhaustively everything in the delivered project that was **not** written by the team.

Libraries. `three.js r128`, loaded from `cdnjs.cloudflare.com` with a single `<script>` tag, is the *only* runtime dependency. It provides: the WebGL context and renderer (`WebGLRenderer`); the scene graph (`Scene`, `Group`, `Object3D`) and its transform composition; `PerspectiveCamera` and projection; the parametric geometry generators (`SphereGeometry`, `CylinderGeometry`, `BoxGeometry`, `TorusGeometry`, `EdgesGeometry`); the standard PBR material (`MeshStandardMaterial`) and its shaders; `InstancedMesh` for batched particle drawing; lights; `CanvasTexture`; `BufferGeometry` for the line visualisations; and the math types (`Vector2/3`, `Color`, `Plane`, `Raycaster`).

That is the complete list. No UI framework (React, Vue), no control-panel library (`dat.GUI`, `lil-gui`, `Tweakpane`), no physics engine (`cannon.js`, `ammo.js`), no math library (`gl-matrix`), no utility library (`lodash`), no bundler (`webpack`, `Vite`, `Rollup`), no package manager, no CSS framework. The control console, the matrix editor, the HUD and the camera controller were all written for this project; in particular we did *not* use `three.js`’s own `OrbitControls`, because it ships as a separate add-on file and our camera has to interoperate with the poke tools, which share the same mouse buttons.

Models, meshes, animation and textures: none imported.

There are no `.obj/.gltf/.fbx` files, no rigs, no keyframe data and no image files anywhere in the project. Every piece of geometry is assembled at run time from `three.js` primitives: particles are one `SphereGeometry(1,10,8)` drawn N times by an `InstancedMesh`; the Grazer is roughly two dozen spheres, cylinders and a torus composed into a hierarchy of `Groups`; the boundary cage is an `EdgesGeometry` of a `BoxGeometry`; the octree, field and lattice visualisations are `BufferGeometry` objects whose vertex buffers we fill by hand every frame. The walk cycle is not an imported clip but a closed-form function of a phase accumulator (§5.6). All three texture maps are painted procedurally into off-screen canvases at start-up and uploaded via `CanvasTexture` (§5.1).

Fonts. The CSS font stacks name Inter, SF Mono, JetBrains Mono and Roboto Mono, but no font is *fetched*: there is no `@font-face` rule and no Google Fonts link, and every stack ends in a generic family (`system-ui`, `sans-serif`, `monospace`). The named fonts are used only if the user’s machine already has them, so the application carries no font dependency.

Conceptual credits. The short-range rule model is our own implementation of the well-known *particle life* / “Clusters” idea popularised by Jeffrey Ventrella [2], credited in the application footer. No code was taken: the original demonstrations are 2D, use all-pairs evaluation and have neither variable mass nor gravity. Our contribution is the extension to 3D, the mass-dependent long-range term with a Barnes–Hut octree, the uniform-grid acceleration of the short-range term, the live visualisations and the piloted creature. Barnes–Hut is the standard published method [1]; the implementation in `js/octree.js` is ours.

Development tools. A text editor (source), the browser developer tools (debugging, profiling, frame timing) and L^AT_EX (this document). No compiler, bundler, package manager or asset pipeline is part of the workflow.

4 Software architecture

The project is deliberately flat: one HTML entry point, one stylesheet, ten JavaScript files, each with a single responsibility.

File	Lines	Responsibility
<code>index.html</code>	& 45 +	Markup for the console, HUD and pilot hint, and the script load order; all presentation: console, HUD, sliders, toggles, matrix grid, responsive rules.
<code>css/style.css</code>	128	
<code>js/config.js</code>	54	CFG (every tunable), Poke (intervention state), PALETTE .
<code>js/scene.js</code>	88	Renderer, camera, lights, boundary cage, poke marker, procedural textures.
<code>js/particles.js</code>	56	Particle buffers, seeding, attraction matrix, InstancedMesh .
<code>js/creature.js</code>	163	Grazer model hierarchy, procedural gait, keyboard piloting.
<code>js/grid.js</code>	38	Uniform grid (counting sort) and the particle-life force kernel.
<code>js/octree.js</code>	124	Barnes–Hut octree: build, mass/COM pass, traversal.
<code>js/simulation.js</code>	282	The integrator, the three debug visualisations, mesh sync.
<code>js/camera.js</code>	71	Orbit/pan/zoom, screen-to-world unprojection, mouse and touch bindings.
<code>js/ui.js</code>	293	Widget factory, control console, matrix editor, impulses, presets.
<code>js/main.js</code>	57	Frame loop, HUD, resize handling, bootstrap.

Dependency order. Because the files are classic scripts sharing one global scope, load order *is* part of the design; each file may use anything declared in the files before it. The order encoded in `index.html` is exactly the dependency graph: `config` → `scene` → `particles` → `creature` → `grid` → `octree` → `simulation` → `camera` → `ui` → `main`. `config.js` is first because every other file reads CFG; `main.js` is last because it bootstraps, and it is the only file that executes significant work at load time. The whole start-up sequence is five lines:

```
allocParticles(N); allocOctree(); rebuildGridDims(); randomizeMatrix(); seedParticles();
buildMesh(); buildCreature(); buildLattice(); buildPanel(); applyCam(); requestAnimationFrame(frame);
```

Data layout: structure of arrays. Particles are **not** objects — there is no `Particle` class. State lives in parallel typed arrays (`px,py,pz, vx,vy,vz` as `Float32Array`; `pType` as `Uint8Array`; `pSize, pMass` as `Float32Array`). This is the single most important performance decision in the project, for three reasons. *Cache locality*: the inner loop streams sequentially through the position arrays instead of chasing pointers into a scattered heap. *No GC pressure*: the arrays are allocated once, at start-up or on a population change, so in steady state the simulation allocates *nothing* per frame and no collection pause ever interrupts the animation. *Predictable types*: `Float32Array` elements cannot be boxed, which lets the JIT emit unboxed floating-point code for the whole loop. The same discipline is applied to the octree (§5.4), a pooled structure of thirteen typed arrays rather than a tree of node objects, and to the gravity-field samples.

5 Technical aspects

5.1 Scene, lighting and procedural textures

The scene is a dark volume with exponential fog (`FogExp2`, density 0.006) matching the background `0x05070d`, so distant particles dissolve instead of ending abruptly at the far plane — a cheap depth cue in a scene that is otherwise a cloud of similar spheres. The camera is a `PerspectiveCamera` (55° FOV, near/far 0.5/2000) and is never manipulated directly: it is a dependent variable of the orbit state (§6.1). The rig is conventional — a cool ambient fill, a directional key and a directional rim that separates silhouettes — plus two accent point lights with finite range and quadratic decay, a teal one animated on a slow circle of radius 40 and a static pink one. The moving teal highlight sweeping across a static clump is often the clearest signal that a structure is three-dimensional rather than a flat sprite cloud; the accent pair can be toggled off.

Three 128×128 maps are painted into off-screen canvases at start-up: an **albedo** map (radial gradient plus 900 faint speckles, kept deliberately *light* because the per-instance colour multiplies it and a dark albedo would mute the type colours); a **normal** map (neutral (128, 128, 255) overlaid with 70 radial gradients whose red and green channels are randomly offset, each blob a small bump, applied at `normalScale` (0.6, 0.6) on particles and a subtler (0.25, 0.25) on the creature); and a **roughness** map (mid-grey with 1400 random 2×2 squares, so a clump of identically-coloured particles does not look like one moulded object). Total cost is a few milliseconds, once.

5.2 Particle representation and instanced rendering

Each particle has position, velocity, a type $t_i \in \{0, \dots, T-1\}$, a radius s_i and a mass $m_i = s_i^3$. Radii are drawn from a distribution deliberately skewed toward small values, $s_i = s_0[(1 - sv) + sv(0.4 + 3.4u^{2.2})]$ with u uniform on $[0, 1]$: the exponent 2.2 biases the variate toward zero, so the population is mostly small particles

with a few large ones which, because $m_i = s_i^3$, carry disproportionate gravitational mass: a handful of heavy seeds act as attractors around which the light majority organises, exactly as in the astrophysical case being evoked. Initial positions are uniform *by volume* inside a sphere, which requires the cube-root inversion of the radial CDF ($\mathbf{r} = \text{Math.cbrt}(\text{Math.random()}) * \mathbf{R}$); omitting the `cbrt` would concentrate particles near the centre and seed a gravitational collapse before the user has touched anything.

All N particles are drawn by a single `InstancedMesh`: one geometry, one material, one draw call. The instance matrix buffer is flagged `DynamicDrawUsage`, and per-instance colour is written only on a type change via `setColorAt`, so only the transform buffer is re-uploaded each frame:

`syncMesh()` rewrites the N matrices each frame using a single reused `Object3D` as a scratch composer, so it allocates nothing. The sphere is only 10×8 segments (80 faces): at $N = 3000$ that is a quarter of a million triangles, which is comfortable, whereas 32×16 would be a million, which is not. The normal map restores the surface detail the coarse tessellation gives up.

5.3 Short-range chemistry and the uniform grid

With $\hat{r} = r/r_{\max}$, β the repulsion-core fraction and $a = A_{t_i t_j}$ the signed matrix entry, the scalar force profile is piecewise linear:

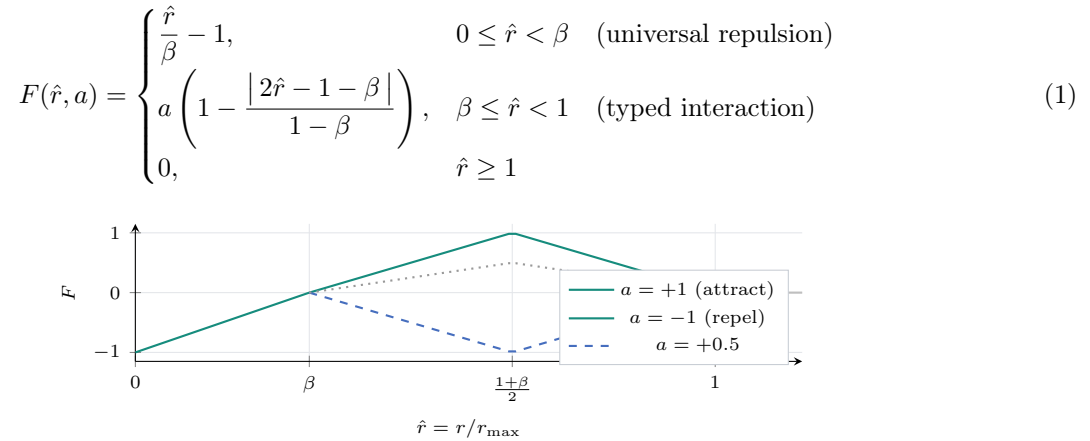


Figure 1: The force profile, drawn with $\beta = 0.3$: a type-independent repulsion below β , a triangular lobe scaled by a peaking at $\hat{r} = (1 + \beta)/2$, and exactly zero beyond $r_{\max}$.

Three properties carry the whole design. **The repulsion core is universal:** below β the profile does not depend on a , so every particle pushes every other away at close range regardless of type, preventing total collapse under mutual attraction — the β slider is effectively an incompressibility control. **The interaction is compactly supported:** F is exactly zero beyond $r_{\max}$, not asymptotically small, which is the precondition for the grid below. **The matrix is asymmetric:** A_{ij} and A_{ji} are independent, so Newton’s third law is deliberately violated and a pair in which red chases green while green flees red self-propels. Physically false, behaviourally essential: it is the source of every chasing and locomotive pattern in the system. The acceleration is

$$\mathbf{a}_i^{\text{chem}} = r_{\max} k \sum_{j: r_{ij} < r_{\max}} F\left(\frac{r_{ij}}{r_{\max}}, A_{t_i t_j}\right) \hat{\mathbf{d}}_{ij}, \quad (2)$$

with k the `forceStrength` slider. The $r_{\max}$ factor keeps the effective force scale roughly constant as the user drags the interaction radius, so changing $r_{\max}$ changes the *geometry* of the patterns rather than simply making everything more violent.

Uniform grid. Since the kernel has compact support, particles are binned into a grid of cell size $r_{\max}$ (resolution capped at 64^3 cells) and only the 27 neighbouring cells are searched — a *complete* search domain, not an approximation. The grid is rebuilt from scratch every frame by a three-pass counting sort into flat typed arrays, which allocates nothing:

```
function buildGrid(){
  cellCount.fill(0);
  for(let i=0;i<N;i++){ const c=cellIndex(px[i],py[i],pz[i]); cellOf[i]=c; cellCount[c]++; }
  let acc=0;
  for(let c=0;c<gCells;c++){ cellStart[c]=acc; acc+=cellCount[c]; }
  cellStart[gCells]=acc;
  const cursor=cellStart.slice(0,gCells);
  for(let i=0;i<N;i++){ const c=cellOf[i]; sortedIdx[cursor[c]]+=i; }
}
```

Pass 1 histograms the cells, pass 2 turns the histogram into a prefix sum, pass 3 scatters indices so that the members of cell c occupy one contiguous slice of `sortedIdx`. The build is $O(N + \text{cells})$ and the layout it produces

means the inner loop walks neighbours contiguously in memory. With roughly constant density the expected query cost is $O(N)$: at $N = 3000$, the difference between 9×10^6 pair tests per frame and a few tens of thousands.

5.4 Long-range gravity: Barnes–Hut octree

Gravity has infinite support, so the grid trick does not apply and exact evaluation is irreducibly $O(N^2)$. We use the Barnes–Hut approximation [1]: a distant *group* of particles is replaced by a single pseudo-particle at its centre of mass. The tree is a pooled structure of thirteen typed arrays indexed by node id — centre `nCx/nCy/nCz`, half-width `nHalf`, mass `nMass`, centre of mass `nComX/Y/Z`, `nBody` (body index if leaf, -1 otherwise), `nInternal`, `nDepth` and `nChild` (8 ids per node) — sized once at $\max(256, 4N + 64)$ nodes on a population change. Rebuilding every frame then costs *zero* allocations: `buildOctree()` resets `nodeCount=0` and re-uses the pool, and children are created lazily so empty space costs nothing.

Bodies are inserted one at a time into a root cube covering the world. The insertion loop handles the standard subtle case: inserting into an occupied leaf requires pushing the existing body down too, and if both bodies land in the same octant the process must recurse.

```
if(nInternal[node]==0){
  if(nBody[node]==-1){ nBody[node]=i; return; } // empty leaf: done
  const e=nBody[node]; nBody[node]=-1; nInternal[node]=1; // occupied: subdivide
  if(nHalf[node]<0.02) return; // coincident-body guard
  const ce=child0f(node, octant(node,px[e],py[e],pz[e])); nBody[ce]=e;
  const ci=child0f(node, octant(node,px[i],py[i],pz[i]));
  if(ci==ce){ node=ci; continue; } // same octant: go deeper
  nBody[ci]=i; return;
}
```

The `nHalf < 0.02` guard is essential: two coincident bodies would otherwise subdivide forever and hang the browser. Below that cell size the body is dropped from the tree for that frame — an invisible error at this scale, and a hard bound on depth; a second guard caps the loop at 64 iterations. A post-order recursion then aggregates $M_{\text{node}} = \sum_c M_c$ and $\mathbf{X}_{\text{node}} = M_{\text{node}}^{-1} \sum_c M_c \mathbf{X}_c$.

For each particle the tree is walked with an explicit stack (a reused `Int32Array(4096)`: recursion would mean a JS call frame per node and no way to bound depth). A node of width s at distance d is accepted as a single pseudo-particle when $s/d < \theta$, evaluated in squared form (`if(s*s < theta2*d2)`) to avoid a square root per test, and is otherwise opened. The contribution of an accepted node is the softened inverse-square law

$$\mathbf{a}_i += \frac{G M_{\text{node}} (\mathbf{X}_{\text{node}} - \mathbf{x}_i)}{(\|\mathbf{X}_{\text{node}} - \mathbf{x}_i\|^2 + \varepsilon^2)^{3/2}}. \quad (3)$$

Softening is not cosmetic: without it two particles passing close together produce $\|\mathbf{a}\| \propto 1/r^2 \rightarrow \infty$ and, with a finite time step, are slingshotted to infinity. The ε^2 term bounds the force at GM/ε^2 ; as in the standard formulation it enters the force but not the opening test. θ is exposed as a slider because it *is* the algorithm’s central dial: $\theta \rightarrow 0$ opens every node and degenerates to exact $O(N^2)$; 0.5 is accurate and slow; 0.8 is the default $O(N \log N)$ compromise; 1.6 is fastest, with visible clumping artefacts. The HUD’s node count makes the trade-off legible in real time. A second entry point, `gravityAccelAt()`, evaluates the same field at an arbitrary point — identical but without the self-exclusion test — and drives the field and lattice visualisations.

5.5 Integration, stability and boundaries

The two force terms, plus the creature field and the user’s intervention field, are summed per particle and integrated with **semi-implicit (symplectic) Euler** — velocity first, then position from the *new* velocity, which is markedly more stable for orbital motion than explicit Euler at the same cost. Friction is an exponential decay expressed through a *half-life*, $\mu = 2^{-\Delta t/T_{1/2}}$, $\mathbf{v} \leftarrow \mu \mathbf{v} + \mathbf{a} \Delta t$: framerate-independent by construction, and a control with a physical meaning (“motion halves in this long”) instead of an arbitrary coefficient. Three further measures keep the system alive: a **time-step clamp** (`simulate(1/60*Math.min(2, rdt*60))`), with the raw frame delta itself clamped at 50 ms) so that returning from a background tab costs a moment of slow motion instead of an explosion; a **velocity clamp** at 90 units/s as a last line of defence; and a **two-pass update**, in which forces for *all* particles are computed from start-of-step positions before *any* position is written — a single fused loop would let particle j see particle i ’s updated position, making the result depend on particle numbering.

Two boundary modes are toggled live. **Reflecting walls** (default) clamp positions to the cube and reverse the normal velocity with restitution 0.6. **Toroidal wrap** makes space a 3-torus, which is more than a position remap: the *forces* must wrap too or structures would tear at the seams. The short-range term therefore uses the minimum-image convention (a separation longer than half the box is replaced by the shorter way round) and the 27-cell sweep wraps modulo `gN` — enabled only when `gN ≥ 3`, below which a cell would be its own neighbour twice and pairs would be double-counted. The position remap uses a floor rather than a conditional subtraction (`px[i] -= worldW*Math.floor((px[i]+h)/worldW)`) so that a particle overshooting by more than one box width still lands correctly instead of flickering at the edge. Gravity is deliberately *not* wrapped: a periodic gravitational field needs an Ewald summation, far beyond a real-time browser demo, so under wrap it is a documented approximation.

5.6 The Grazer: hierarchical modelling and procedural gait

The creature demonstrates the classical hierarchical-modelling requirement: a tree of transforms in which each child's world transform is the composition of its own local transform with all of its ancestors'. We use three.js's scene graph as the transform tree, but the hierarchy, the joint layout and the motion are ours. Depth from root to foot is five levels:

```
root (Group) → { body+belly | face → eye → (sclera, pupil, glints) | antenna×2 → (stalk, bulb) |
hip×4 → thighPivot → knee → shinPivot → foot }
```

Rotating `thighPivot` carries the knee, shin and foot with it; rotating `shinPivot` carries only the foot. The subtlety in building an articulated limb from primitives is that a `CylinderGeometry` is generated centred on the origin and aligned with $+y$, which is the wrong frame for a bone — rotating such a mesh about a joint would swing it about its middle. The geometry is therefore baked into the correct frame once, at build time:

```
const thighGeo = new THREE.CylinderGeometry(0.2, 0.16, L1, 10);
thighGeo.rotateZ(-Math.PI/2); // lie along +x instead of +y
thighGeo.translate(L1/2, 0, 0); // span 0..L1, so the joint sits at the origin
const knee = new THREE.Group(); knee.position.set(L1,0,0); thighPivot.add(knee);
```

Because the knee is a *child* of the thigh's pivot it inherits every rotation of the thigh automatically: the segments cannot come apart, not because the animation is careful but because the hierarchy makes separation unrepresentable. The four hips sit on the body surface at radius 2.45 and are rotated so each limb's local $+x$ points radially outward, so thighs sweep away from the body and never intersect it.

The walk cycle is a closed-form function of a phase accumulator — there is no keyframe data anywhere:

```
const gaitSpeed = 5 + moving*0.5; // legs cycle faster when moving faster
c.phase += dt*gaitSpeed;
for(const leg of c.legs){
  const ph = c.phase + leg.phase; // per-leg constant offset 2*pi*k/4
  const lift = Math.max(0, Math.sin(ph)), swing = Math.cos(ph);
  leg.thighPivot.rotation.z = 0.18 + swing*0.24 - lift*0.10;
  leg.thighPivot.rotation.y = swing*0.16;
  leg.shinPivot.rotation.z = -1.7 + lift*0.5;
}
```

The four constant phase offsets $2\pi k/4$ distribute the legs evenly around the cycle so the gait reads as a coordinated walk rather than four independent twitches. The half-wave rectification $\max(0, \sin)$ distinguishes the two halves of the cycle: during swing the leg lifts, during stance `lift` is pinned to zero so the foot stays planted — a lift term of $\sin$ alone would make the leg burrow into the ground on the downstroke. The constant -1.7 rad of knee bend gives the silhouette its insect-like character. Three secondary motions layer on top, all closed-form: a hover bob $\sin(2.2t)$ on the root, springy antennae leaning back in proportion to speed, and a stochastic blink that squashes the eye's y scale for 140 ms then re-arms after 2.5–5 s. Heading is critically damped rather than snapped (`rotation.y += (heading-rotation.y)*min(1,dt*6)`), so turns ease instead of popping.

A solid body with a field. The Grazer is not a ghost. After integration, particles inside its bounding radius are projected onto the surface and given an outward impulse $-1.4 v_n$, leaving a residual $-0.4 v_n$ (restitution 0.4); the impulse is applied only when $v_n < 0$, i.e. only to particles moving *into* the surface, without which a particle sliding along it would be kicked repeatedly. The proxy is a sphere rather than the visual mesh — per-triangle collision against an articulated body would cost more than the rest of the frame for an imperceptible difference. Within a radius R the creature also applies a quadratic-falloff radial force plus a tangential swirl,

$$\mathbf{a} += \underbrace{\sigma S \frac{(1 - d/R)^2}{d}}_{\text{attract/repel}} \mathbf{d} + \underbrace{0.25 S \frac{(1 - d/R)}{d}}_{\text{swirl about } y} (-d_z, 0, d_x), \quad (4)$$

with $\sigma = \pm 1$ the mode and S the strength slider. The quadratic falloff makes the field vanish smoothly at $r = R$ instead of cutting off with a discontinuity visible as a hard shell, and the swirl term is why gathered particles orbit the creature instead of piling into a static ball — it converts pure radial infall into an accretion disc, which is far more legible as motion. The aura sphere is scaled to exactly R , so the visible bubble is an honest depiction of the field's reach, and its colour tracks the mode (cool blue for attract, warm orange for repel).

5.7 Visualising the invisible

Octree wireframe. A recursive walk emits the twelve edges of each occupied node's cube into a `BufferGeometry`, capped at depth 6 and skipping empty leaves. The result makes Barnes–Hut adaptivity immediately obvious: cells are large in empty space and subdivide finely exactly where mass concentrates.

Gravitational field streamlines. The field is sampled on an $n \times n \times n$ lattice ($n = \text{fieldN}$, up to $20^3 = 8000$ samples) via `gravityAccelAt`. Each sample is drawn as two segments sharing one geometry: a faint full-length vector showing structure, and a bright pulse travelling along it toward mass, parameterised by $\mathbf{f} = ((\mathbf{t} \cdot \text{speed} + \text{FLD.ph}[\mathbf{k}]) \% 1 + 1) \% 1$ with a glow $0.45 + 0.55 \sin(\pi \mathbf{f})$. The per-sample phase `FLD.ph[k]` is the linear function

$0.05x + 0.045y + 0.04z$ of position: because the coefficients are unequal the wave sweeps diagonally across the volume and never resynchronises into a distracting pulse-in-unison. Colour interpolates teal $\rightarrow$ orange with strength, magnitudes are normalised against the frame maximum, and samples below 4% collapse to a degenerate point rather than being culled, keeping the vertex count constant so the buffer is never reallocated. Rendering is additive with `depthWrite:false`. Crucially, the sample *directions* are rebuilt only every fourth frame while the pulse is redrawn every frame: the expensive part (8000 octree traversals) runs at 15 Hz, the cheap part at 60 Hz, and the eye cannot tell.

Deforming space lattice. A 9^3 grid of nodes joined by ~ 1944 segments is displaced from its rest position by the local gravitational acceleration ($\times 7$) and by the user’s active poke field, capped at 9 units so a violent warp bends space instead of shredding the lattice. Rest positions are stored separately from current ones, so the deformation is always computed from the undeformed state and never accumulates drift. It is the intended companion to the Warp tool.

5.8 The frame loop and performance

```
function frame(now){
  const rdt = Math.min(0.05, (now-last)/1000); last=now; let octRoot = -1;
  if(CFG.running) octRoot = simulate(1/60*Math.min(2, rdt*60)); // physics
  pilot(rdt); animateCreature(rdt, t); syncMesh(); // input, gait, instances
  if((CFG.showField||CFG.showLattice) && CFG.gravity && octRoot<0) octRoot = buildOctree();
  if(CFG.follow && creature.root) cam.target.lerp(creature.root.position, 0.05);
  applyCam(); renderer.render(scene, camera); requestAnimationFrame(frame);
}
```

`simulate()` returns the octree root it built so the visualisations reuse it; the tree is constructed independently only when the simulation is paused but a visualisation still needs it. The follow camera uses `lerp(...,0.05)` rather than assignment, giving a damped chase that lags slightly — an instant snap would make the whole world jitter with the gait’s hover bob.

Stage	Complexity	Notes
Grid build	$O(N + \text{cells})$	Counting sort, allocation-free
Short-range forces	$O(N)$ expected	27-cell sweep at constant density
Octree build	$O(N \log N)$	Pooled nodes, allocation-free
Gravity traversal	$O(N \log N)$	Iterative, shared explicit stack
Mesh sync / render	$O(N)$ / 1 draw call	One reused <code>Object3D</code> ; <code>InstancedMesh</code>
Field / lattice	$O(n^3 \log N)$ / $O(9^3 \log N)$	Only when enabled; field throttled to every 4th frame

The HUD exposes the live picture: frame rate, particle count, pairs evaluated per frame, octree node count and the current mode (*chem* or *2-scale*). The steady-state allocation rate is zero, so no GC pause ever interrupts the animation.

6 Implemented interactions

The application is interactive in five registers: the user can *look* (camera), *touch* (drag tools), *strike* (one-shot impulses), *inhabit* (piloting the creature) and *legislate* (editing the rules).

6.1 Camera control

The camera is driven by a hand-written spherical orbit state (`{target, theta, phi, radius}`), from which the position is recomputed each frame and pointed at the target. Left-drag orbits ($\theta \leftarrow 0.005\Delta x$, $\phi \leftarrow 0.005\Delta y$) when the Orbit tool is active; right-drag orbits *always*, whatever tool is selected, so the user is never trapped in a tool (the canvas context menu is suppressed to make this possible); middle-drag or `[Shift]+left-drag` pans; the wheel zooms; touch gives one-finger orbit and two-finger pinch. Four details matter: ϕ is clamped to $[0.08, \pi - 0.08]$, because reaching a pole exactly makes the view direction parallel to the up vector and `lookAt` produces a degenerate basis that flips the camera; zoom is *multiplicative* ($\times 1.08$ per notch, clamped to $[20, 500]$), since an additive zoom crawls when far away and overshoots through the subject when close; pan speed scales with radius so a given mouse movement always covers the same fraction of the screen; and panning silently cancels follow mode, without which the camera would fight the user’s input and lose — which reads as a bug rather than a feature.

6.2 Direct manipulation: the drag tools

Four tools share the left mouse button: **Orbit** (default, no effect on the simulation), **Push** (radial outward force inside the brush sphere, blasting a cavity in the soup), **Pull** (radial inward, gathering particles into the cursor) and **Warp** (a violent composite of swirl, collapse and axial compression producing a lensing/folding effect). The brush radius is a slider (5–60 units) and a master **Intensity** multiplier ($0.2\text{--}6\times$) scales the tools *and* every impulse below. A translucent sphere with a rotating ring marks the brush in the scene, sized to R and coloured by mode, so the region of influence is never guesswork.

The mouse gives a 2D position but the force needs a 3D one, so `screenToWorld()` converts pixels to NDC,

builds a ray with `Raycaster.setFromCamera` and intersects it with the plane through the orbit target whose normal is the camera’s view direction — the plane the user perceives as “the screen, pushed into the scene”. Because that plane always faces the camera, the tool acts at the depth the user is looking at and behaves identically from every angle; the function reuses five module-level scratch objects and allocates nothing, which matters because it runs on every `mousemove`. With d the distance to the poke centre, R the brush radius, $\text{fall} = 1 - d/R$ and I the intensity:

$$\text{Push/Pull: } \mathbf{a} += \pm 420 I \frac{\text{fall}^2}{d} \mathbf{d} \quad (5)$$

$$\text{Warp: } \mathbf{a} += \underbrace{620 I \text{fall}^2 (\hat{\mathbf{n}} \times \mathbf{d})}_{\text{swirl}} - \underbrace{520 I \frac{\text{fall}^2}{d} \mathbf{d}}_{\text{collapse}} - \underbrace{300 I \text{fall} (\mathbf{d} \cdot \hat{\mathbf{n}}) \hat{\mathbf{n}}}_{\text{axial compression}} \quad (6)$$

The swirl axis $\hat{\mathbf{n}}$ is captured as the camera’s forward vector at the moment the drag begins, so the vortex always spins in the plane of the screen and the user sees a whirlpool face-on rather than edge-on. The quadratic falloff again avoids a hard edge at $r = R$.

6.3 One-shot impulses

Six buttons apply an instantaneous change to every particle, centred on the current orbit target — i.e. on whatever the user is looking at. Unlike the drag tools they modify velocity (or, for Fold, position) directly rather than adding a force. **Shockwave**: radial outward kick $40I/(1 + 0.05d)$, an explosion decaying with distance. **Implode**: radial inward, $45I/(1 + 0.04d)$. **Vortex**: tangential kick $35I$ about the vertical axis through the epicentre, plus a small random vertical component that keeps the disc from becoming perfectly flat. **Warp**: swirl about the view axis plus inward collapse, both $\propto 1/(1 + 0.03d)$. **Jitter**: a uniform random kick up to $40I$ per axis — thermal noise, useful for shaking a structure out of a local minimum. **Fold** is the most interesting, because it is a transformation of space rather than a force: with $\hat{\mathbf{n}}$ the view axis, every particle on the far side of the plane is mirrored onto the near side, *position and velocity*:

```
if(along>0){
  px[i]-=2*along*ax; py[i]-=2*along*ay; pz[i]-=2*along*az; // mirror position
  const vn = vx[i]*ax + vy[i]*ay + vz[i]*az;
  vx[i]-=2*vn*ax; vy[i]-=2*vn*ay; vz[i]-=2*vn*az; // mirror velocity
}
```

Reflecting the velocity as well as the position is what makes the result coherent: a mirrored particle continues along the mirrored trajectory, so folding a rotating galaxy produces two counter-rotating halves that then collide, rather than a pile of particles moving incoherently.

6.4 Piloting the Grazer

W/**S** move forward/backward along the creature’s heading; **A**/**D** strafe *and* turn the heading (± 1.8 rad/s); **Q**/**E** ascend/descend; **Shift** boosts acceleration $2.2\times$; **H** shows or hides the console. Movement is force-based rather than positional: the input assembles an acceleration in the creature’s local frame, which is integrated with heavy exponential damping.

```
if(active) accel.normalize().multiplyScalar(60*boost); // normalize: no faster diagonals
c.vel.addScaledVector(accel, dt);
c.vel.multiplyScalar(Math.pow(0.02, dt)); // damping, framerate-independent
c.root.position.addScaledVector(c.vel, dt); c.root.position.clamp(min, max);
```

The `normalize()` before scaling is what stops diagonal movement from being $\sqrt{2}$ times faster than axis-aligned movement — the classic bug — and the damping uses the same framerate-independent exponential form as the particle friction. The result has weight: the creature accelerates and coasts rather than teleporting, which also means it stirs the soup as it goes. A hint bar listing the controls fades in while the creature moves and out when it stops, so the controls are discoverable without a permanent overlay. A **Follow camera** toggle makes `cam.target` chase the creature with a 5% lerp per frame; as noted, panning cancels it.

6.5 Editing the rules: the attraction matrix

The most consequential interaction is also the smallest widget. The editor is a $T \times T$ grid of coloured cells with row and column headers of type swatches; **rows act on columns**, so cell (i, j) is the force type i feels toward type j . Colour encodes value (green attraction, red repulsion, near-black neutral, intensity $\propto |A_{ij}|$); clicking a cell cycles it through $+1 \rightarrow +0.5 \rightarrow 0 \rightarrow -0.5 \rightarrow -1 \rightarrow +1$, snapping to the nearest step first so a cell holding a random value like 0.37 from *Reroll* still cycles sensibly; a tooltip reports the exact value (type 2 $\rightarrow$ 3 (-0.50)); and changes are instantaneous, because **A** is read directly by the force kernel every frame — no apply button, no reseed. The asymmetry is the pedagogical point and the editor is built to make it discoverable: setting $A_{01} = +1$ and $A_{10} = -1$, one click each, makes type 0 chase type 1 while type 1 flees, and the pair immediately begins to self-propel across the volume; making the matrix symmetric instead produces static clumps. A user can find this out in ten seconds without reading anything.

6.6 The control console and presets

The console is a collapsible panel of nine groups, all collapsed by default so the interface is not a wall of sliders, and every control carries a one-line explanation underneath it. It is built by a small widget factory (`slider`, `toggle`, `btn`, `group`, `withDesc`) — around 30 lines of DOM code that replaces a UI dependency.

Group	Controls
Simulation	Run/Pause (freezes physics, rendering and camera stay live); Reset (re-scatters particles, keeps the rules); Reroll (new random matrix, keeps the particles); the $T \times T$ attraction-matrix editor
Chemistry	Force strength (0.2–8); Interaction radius $r_{\max}$ (3–20, rebuilds the grid live); Repulsion core β (0.05–0.6); Friction half-life (0.01–0.2: syrupy $\rightarrow$ lively)
Gravity	Enable (switches the HUD to <i>2-scale</i>); Strength G (0–4); θ (0.2–1.6); Softening (0.5–6); Show octree; Show gravity field; Field density (3–20 samples per axis)
Population	Particles (200–3000, reallocates on release); Types (2–8, resizes the matrix); Size variance (0–1)
Intervene	Intensity (0.2–6 $\times$); Tool: Orbit/Push/Pull/Warp; Brush radius (5–60); six impulse buttons
World & render	World size (40–160); Wrap edges (torus); Accent point lights; Show space lattice
Grazer	Show creature (also disables its field); Follow camera; Field Attract/Repel; Field radius (6–40); Field strength (0–400, 0 = harmless wanderer)
Presets	Nine one-click regimes

Live reconfiguration. Several controls rebuild live state, which is where the interesting engineering hides. *Particle count* reallocates all nine typed arrays, the octree pool and the grid, then reseeds and rebuilds the `InstancedMesh` (whose instance count is fixed at construction); it is bound to `change` rather than `input`, so it fires once on release instead of on every pixel of the drag. *Interaction radius* changes the grid cell size, so `rebuildGridDims()` runs on every step — cheap, since only index arrays are reallocated. *World size* is the subtlest: rather than clipping particles into the new volume it *scales all positions* by the ratio of new to old half-width, so structures survive the resize instead of being crushed against the walls.

Presets. Nine presets reconfigure rules, gravity and size distribution in one click, each a genuinely different regime: **Cells** (5 types, no gravity, $r_{\max} = 10$ — soft membrane-like blobs), **Chase** (4, none, 12 — asymmetric hunting with moving trails), **Galaxies** (3, $G = 1.6$, 8 — spiral orbiting clumps), **Web** (6, none, 9 — connected filament network), **Orbits** (3, $G = 2.4$, 6 — slow stable orbits), **Worms** (3, none, 14 — wriggling chains), **Crystal** (4, none, 8 — rigid lattice-like solid), **Nebula** (5, $G = 0.7$, 11 — diffuse swirling clouds) and **Swarm** (3, none, 13 — flocking drift). Each also sets β , the friction half-life and the size variance, rerolls the matrix, reseeds and rebuilds the panel. Because the matrix is rerolled, clicking the same preset twice gives two different worlds *of the same character*: the preset fixes the regime, not the outcome.

7 User manual

Running it. Open `index.html` in any modern browser — double-clicking the file is enough; no server, no installation, no build step. An internet connection is needed the first time, to fetch `three.js` from the CDN; to work fully offline, download `three.min.js` and point the first `<script src>` in `index.html` at the local copy. The simulation starts immediately with 1000 particles, 5 types, a random rule matrix and gravity off.

The screen. On the **left** is the control console: click a section header to expand it, click `◀` (or press `[H]`) to hide the panel and the `☰` tab to bring it back. **Top right** is the HUD: frame rate, particle count, pairs evaluated per frame, octree node count and the mode (*chem* = chemistry only, *2-scale* = chemistry + gravity). **Bottom centre** is the pilot hint bar, which appears while the Grazer is moving. In the **centre** is the simulation itself, inside a faint wireframe cage marking the world boundary.

Input	Action	Input	Action
Left-drag	Orbit, or the active tool	WASD	Pilot the Grazer
Right-drag	Orbit (always)	QE	Grazer up / down
Middle-drag / <code>[Shift]</code> +left	Pan	Shift	Boost
Scroll	Zoom	H	Toggle the console
One finger / two fingers	Orbit / pinch-zoom		

A guided tour (about ten minutes). **1. Watch first:** let the default matrix run for thirty seconds and orbit around it — whatever appears was not designed, it is the matrix expressing itself. **2. Change one rule:** in *Attraction rules* click a cell until it is bright green and its mirror cell until it is bright red, then watch those two colours chase each other; this asymmetry is the engine of every moving pattern in the system. **3. Try the**

presets: compare *Crystal* (rigid) with *Worms* (crawling chains) — same code, same particles, different numbers. **4. Turn on gravity:** click *Galaxies*, watch the HUD switch to *2-scale*, then enable *Show octree* — cubes are large in empty space and subdivide tightly around the clumps, which is why the frame rate survives. **5. Break the octree on purpose:** drag θ from 0.8 to 1.6 and watch the node count collapse and the frame rate climb, then down to 0.3 for the opposite; you are looking directly at the accuracy/speed trade-off. **6. See the field:** enable *Show gravity field* and raise *Field density* to about 12. **7. Bend space:** enable *Show space lattice*, select **Warp** and drag through a clump, then raise *Intensity* to 4 $\times$ and try again. **8. Fold the universe:** click *Fold* and watch the two halves collide. **9. Go for a swim:** set *Field: Attract*, *Field strength* ≈ 300 , enable *Follow camera* and pilot with **W A S D** — particles gather into an orbiting disc dragged behind the Grazer; switch to *Repel* to plough a tunnel instead. **10. Push the limits:** raise *Particles* to 3000 and watch the HUD, then find out whether θ , the field density or the visualisations buy the most frame rate back.

Symptom	Cause and remedy
Black screen, nothing renders	three.js failed to load from the CDN. Check the network, or switch to a local copy of three.min.js .
Low frame rate	Lower <i>Particles</i> ; raise θ ; lower <i>Field density</i> ; disable the octree/field/lattice visualisations.
Everything collapses to a dot	G too high or <i>Repulsion core</i> too low: raise β , lower G , or raise <i>Softening</i> . If instead particles fly apart, <i>Force strength</i> or <i>Intensity</i> is too high.
Nothing happens when I drag	The active tool is <i>Orbit</i> . Select Push, Pull or Warp — or right-drag to orbit.
The camera will not follow	Follow mode disables itself whenever you pan. Re-enable it in <i>Grazer</i> .
No creature visible	<i>Show creature</i> is off, or it has been piloted out of view: enable <i>Follow camera</i> to find it.

8 Limitations, future work and conclusions

Each limitation below is a considered trade-off rather than an oversight. The simulation is **single-threaded and CPU-bound**: beyond roughly 3000 particles the force evaluation, not the rendering, is the bottleneck, and the fixes — a Web Worker with a **SharedArrayBuffer**, which needs COOP/COEP headers and therefore a server, or GPU-side integration — would either break the `file://` promise or require a substantial rewrite. **Gravity is not periodic**: under *Wrap edges* the short-range term uses the minimum image but gravity still sees the non-wrapped configuration, since a correct treatment needs an Ewald summation. **The creature's collision proxy is a sphere**, so particles bounce off an invisible ball rather than off the legs, and **the gait is not foot-locked**, so a slight skate remains at high speed — proper foot-locking needs inverse kinematics. **Momentum is not conserved**, by design: the asymmetric matrix violates Newton's third law, which is the entire point of the chemistry model but does mean there is no conserved quantity to validate against. **Bodies are dropped at extreme density** by the octree's coincident-body guard, a small bounded error preferable to an infinite subdivision that would hang the browser, and **instance count is fixed at construction**, so changing the particle count rebuilds the **InstancedMesh** — which is why that control fires on slider release. Natural extensions: GPU-side integration; an IK solver with foot locking; multiple creatures flocking among themselves; saving matrices as shareable URLs; and soft-body chains built on the existing neighbour grid.

PRIMORDIA sets out to demonstrate that complexity does not require complicated rules. Two force laws — a piecewise-linear typed kernel and a softened inverse-square law — acting at two different spatial scales are enough to produce cells, chains, filaments, orbits, spirals and swarms, none of which appears anywhere in the source as an explicit construct. What makes this interactive rather than merely correct is the pairing of each law with the acceleration structure suited to its scale: a uniform grid, valid precisely because the short-range kernel has compact support, and a Barnes–Hut octree, valid because a distant cluster is well approximated by its centre of mass. Both are built on pooled typed arrays, so the steady-state allocation rate is zero. The user-facing payoff of that work is the θ slider and the octree visualisation: the approximation is not hidden behind the curtain, it is on stage. The project also meets its asset-free constraint completely — every vertex, texel and joint angle is computed at run time, and the whole application is one HTML file, one stylesheet and ten JavaScript files with a single third-party dependency.

References

- [1] J. Barnes and P. Hut, *A hierarchical $O(N \log N)$ force-calculation algorithm*, Nature, vol. 324, pp. 446–449, 1986.
- [2] J. Ventrella, *Clusters — artificial life with simple particle interaction rules*. <http://www.ventrella.com/Clusters/>
- [3] three.js — a JavaScript 3D library, revision 128. <https://threejs.org>
- [4] S. J. Aarseth, *Gravitational N-Body Simulations: Tools and Algorithms*, Cambridge University Press, 2003. (Softening and time-step stability.)